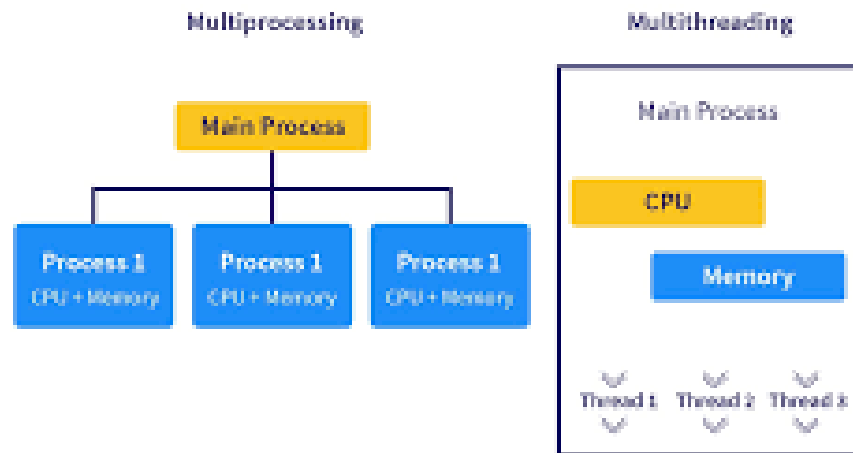

Welcome & Module Induction

- **What we learned previously:** In Module 11, we learned how to serve our code to the world using FastAPI and Uvicorn.
- **What we are learning today:** We are learning how to do more than one thing at a time. We will explore **Multi-threading** (for tasks that wait), **Multiprocessing** (for heavy math), and the infamous **GIL** (the internal speed limit of Python).
- **Why this will be helpful:** Standard Python code is "Synchronous," meaning it does one task at a time. If you are downloading 100 images for a dataset, your code waits for Image 1 to finish before starting Image 2. Today, you will learn how to make your computer work in parallel, cutting your processing time from minutes to seconds.

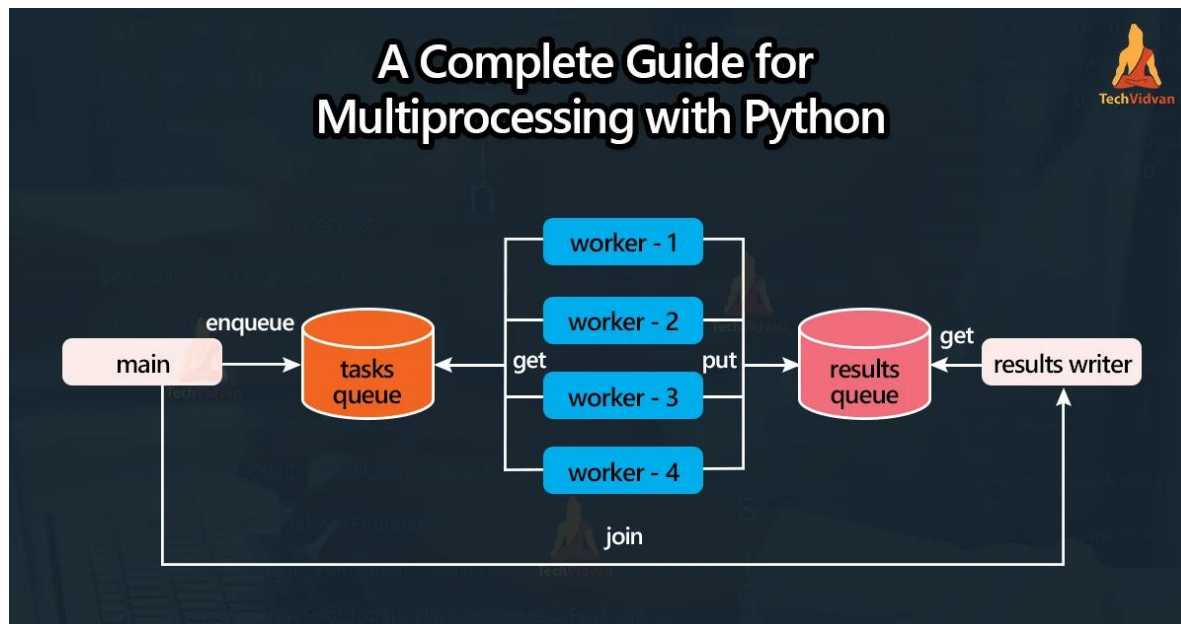
Multi-threading (The "Waiting" Room)



Scaler
Topics

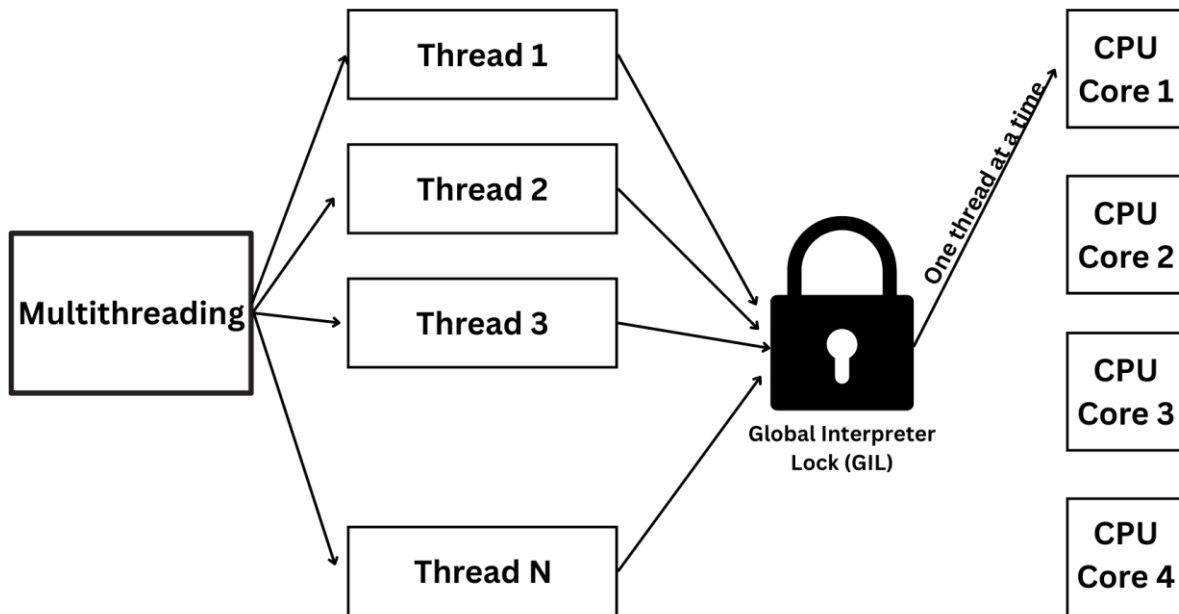
- **The Analogy:** Imagine a waiter in a restaurant. The waiter takes an order, gives it to the kitchen, and then immediately moves to the next table while the food is cooking. They don't just stand there waiting.
- **The Use Case:** Threading is perfect for **I/O-bound tasks** (Input/Output). This includes downloading files, querying a database, or waiting for an API response.
- **How it works:** Python switches between "threads" so fast that it looks like they are running at once, even though they are technically sharing one CPU core.

Multiprocessing (The "Factory" Floor)



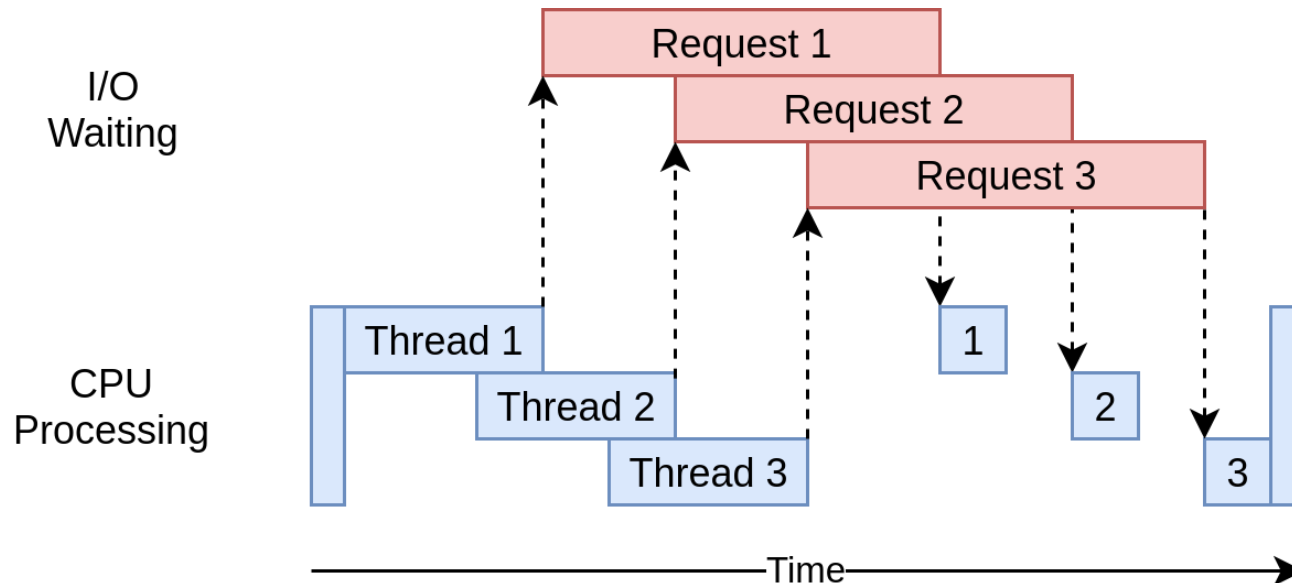
- **The Analogy:** Instead of one waiter, imagine you hire four separate chefs, each with their own stove and kitchen. They all cook different meals at the exact same time.
- **The Use Case:** Multiprocessing is for **CPU-bound tasks**. This includes heavy mathematical calculations, image processing, or training AI model components.
- **How it works:** It bypasses Python's limitations by launching entirely separate instances of Python on every core of your CPU (e.g., using all 8 cores of an i7 processor).

The GIL (The Traffic Cop)



- **What is it?** The Global Interpreter Lock (GIL).
- **The Conflict:** To prevent your computer's memory from getting corrupted, Python only allows *one* thread to hold control of the Python interpreter at a time.
- **The Impact:** This is why Multithreading doesn't make math faster in Python—the GIL acts like a "Traffic Cop" that only lets one car through the intersection at a time. To truly go fast with math, we must use Multiprocessing to create new "intersections."

Advanced Concurrency Scenarios



- **Scenario A (I/O Bound):** You are building a monitoring dashboard for 10 different AI microservices. You need to check the status of all of them every minute.
- *Without Threading:* You wait for Service A, then Service B... the total time is the sum of all wait times.
- *With Threading:* You fire all requests at once. The total time is only as long as the *slowest* single service.

Advanced Concurrency Scenarios

- **Scenario B (CPU Bound):** You have a dataset of 1,000 thermal images that need to be grayscaled and resized before entering a YOLO model.
- *Without Multiprocessing:* One CPU core works on Image 1, then Image 2. It is a slow, sequential crawl.
- *With Multiprocessing:* Your 8-core CPU handles 8 images at the exact same time. The processing speed increases by nearly 800%.

